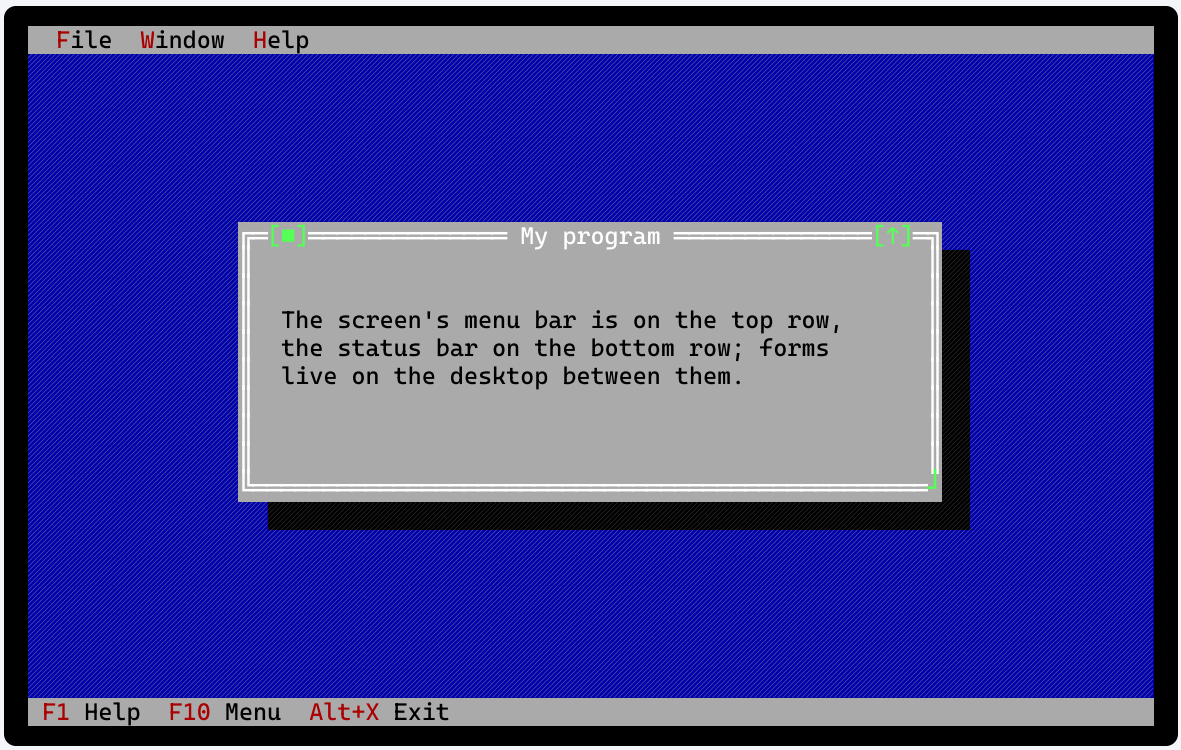
FPTUI User Guide

FPTUI is a text-mode user interface for Free Pascal on 32-bit DOS (go32v2). Forms and controls follow Lazarus (and FPTEGL's DUI): you make a `TForm` descendant, put `TButton`s, `TEdit`s and friends on it, hook up `OnClick` and friends, and call `Application.Run`. Everything is drawn with text cells in the Turbo Vision / Technojock's toolkit style.

The screens in this guide are not mock-ups. `GUIDESHOT.PAS` builds each example with the code shown and records what the screen shows.



The whole screen: the menu bar, the desktop with a form, the status bar.

Getting started

Put the TUI units (`TUI*.PAS`, `TUI.INC`) where the compiler finds them and build for DOS:

```
fpc -Tgo32v2 -O2 myprog.pas
```

Like every go32v2 program it needs a DPMI host: `CWSDPMI.EXE` on the `PATH` (Free Pascal's go32v2 install has one) or `HDPMI32 -r`.

A program is a project file and one unit per form, just as in Lazarus. The project:

```
program MyProg;

{$mode objfpc}{$H+}

uses
  TUICtrl, MainForm;

begin
  Application.Title := 'My program';
  Application.Initialize;
  Application.CreateForm(THelloForm, HelloForm);
  Application.Run;
end.
```

The form unit. The constructor makes the controls here; they can also come from an `.lfm` file, as in Lazarus (see Forms from .lfm files):

```
unit MainForm;

{$mode objfpc}{$H+}

interface

uses
  Classes, SysUtils, TUICtrl, TUIStd;

type
  THelloForm = class(TForm)
    NameLabel: TLabel;
    NameEdit: TEdit;
    GreetButton: TButton;
    procedure GreetButtonClick(Sender: TObject);
  public
    constructor Create(AOwner: TComponent); override;
  end;
```

```
var
  HelloForm: THelloForm;

implementation

constructor THelloForm.Create(AOwner: TComponent);
begin
  inherited Create(AOwner);
  Caption := 'Hello';
  SetBounds(2, 1, 40, 8);              { Left, Top, Width, Height in cells }

  NameEdit := TEdit.Create(Self);
  NameEdit.Parent := Self;
  NameEdit.SetBounds(9, 1, 26, 1);
  NameEdit.Text := 'World';

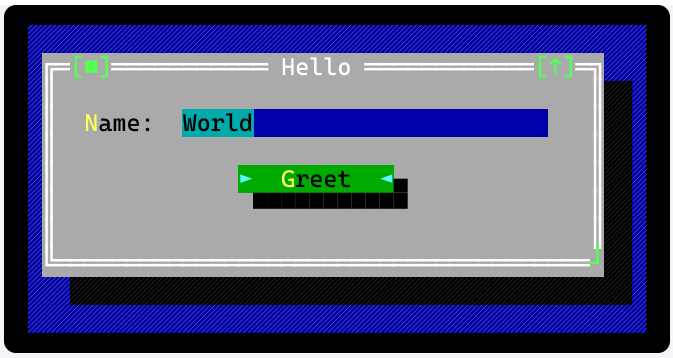
  NameLabel := TLabel.Create(Self);
  NameLabel.Parent := Self;
  NameLabel.Left := 2;
  NameLabel.Top := 1;
  NameLabel.Caption := '&Name:';
  NameLabel.FocusControl := NameEdit;  { Alt+N goes to the edit field }

  GreetButton := TButton.Create(Self);
  GreetButton.Parent := Self;
  GreetButton.SetBounds(13, 3, 12, 2);
  GreetButton.Caption := '&Greet';
  GreetButton.Default := true;         { Enter anywhere on the form }
  GreetButton.OnClick := @GreetButtonClick;

  ActiveControl := NameEdit;
end;

procedure THelloForm.GreetButtonClick(Sender: TObject);
begin
  ShowMessage('Hello, ' + NameEdit.Text + '!');
end;

end.
```



The Hello form: a label with an access key, an edit field, the default button.

The first form created is the main form; `Application.Run` shows it, and closing it ends the program. Every control is owned by the form (`Create(Self)`) and freed with it; `Parent` says where it sits.

`TUIDEMO.PAS` (with `MAINFRM.PAS` and the other `*FRM.PAS` units) shows every control; `TUITEST.PAS` drives the demo with posted keys and mouse clicks and checks the results.

Cells and colours

Positions and sizes are character cells: `Left`, `Top`, `Width`, `Height` count columns and rows.

- A form's `Left` / `Top` are screen cells and its `Width` / `Height` include the frame. `ClientWidth` / `ClientHeight` are the inside.
- A control's `Left` / `Top` are relative to its parent's client area (the inside of the frame, below a window's menu bar).
- Buttons are two rows high: the face and the shadow.

`TColor` is one of the 16 text colours. The LCL names map onto them, and the text-mode names are there too:

Value	LCL name	Text mode name	Value	LCL name	Text mode name
0	clBlack		8	clGray	clDarkGray
1	clNavy		9	clBlue	clLightBlue

Value	LCL name	Text mode name	Value	LCL name	Text mode name
2	clGreen		10	clLime	clLightGreen
3	clTeal	clCyan	11	clAqua	clLightCyan
4	clMaroon		12	clRed	clLightRed
5	clPurple	clMagenta	13	clFuchsia	clLightMagenta
6	clOlive	clBrown	14	clYellow	
7	clSilver	clLightGray	15	clWhite	

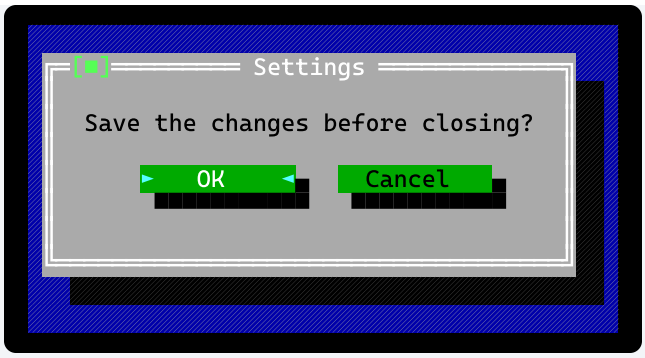
`clDefault` means "the theme decides". `Color` is the background, `Font.Color` the text:

```
StatusLabel.Font.Color := clBlue;
InfoForm.Color := clCyan;
```

The theme (`Theme` in `TUICTRL`) holds every colour the controls use; `SetColorTheme` and `SetMonoTheme` fill it (mono is picked on MDA/ Hercules screens).

Forms

```
F := TForm.CreateNew(nil);           { a form without a class of its own }
F.Caption := 'Settings';
F.SetBounds(0, 0, 38, 8);
F.Position := poScreenCenter;         { poDesigned, poScreenCenter, poMainFormCent
F.BorderStyle := bsDialog;            { bsSizeable (default), bsSingle, bsDialog,
```



A dialog (BorderStyle bsDialog) with OK as the Default and Cancel as the Cancel button.

To	Use
show a window	`F.Show` (it comes to the front and is active)
hide it	`F.Hide` or `F.Close`
run it as a dialog	`if F.ShowModal = mrOk then ...`
end a dialog	set `F.ModalResult`, or a button's `ModalResult`
free it after closing	`OnClose`: `CloseAction := caFree`
maximize	`F.WindowState := wsMaximized`
catch keys first	`F.KeyPreview := true` and `OnKeyDown`

A dialog the usual way:

```
OkButton.ModalResult := mrOk;          { a click closes the dialog with mrOk }
OkButton.Default := true;              { Enter }
CancelButton.ModalResult := mrCancel;
CancelButton.Cancel := true;           { Esc }
if F.ShowModal = mrOk then SaveSettings;
F.Free;
```

Events: `OnCreate` (after the constructor), `OnShow`, `OnHide`, `OnActivate`, `OnDeactivate`, `OnClose`, `OnCloseQuery` (`CanClose := false` keeps it open), `OnResize`, `OnKeyDown`, `OnKeyPress`.

`bsSizeable` forms can be resized and maximized: drag the lower right corner, click `[↑]` or double click the title; from the keyboard Ctrl+F5, then arrows move and Shift+arrows size. `Constraints.MinWidth` etc. limit it. Drag a window by its title; `[■]` closes it.

Forms from .lfm files

A form's controls can come from an `.lfm` file instead of the constructor. The syntax is Lazarus's text `.lfm` (`object Name: TClass`, `Property = value`, nested objects, `end`), and it maps one to one onto FPTUI: every `Left`, `Top`, `Width` and `Height` is in **character cells** (columns and rows), exactly as in code. Nothing is scaled.

The form unit only lists the controls and the event handlers; no constructor is needed:

```
unit LfmDemo;

{$mode objfpc}{$H+}

interface

uses
  Classes, SysUtils, TUICtrl, TUIStd, TUIList, TUIMenu, TUIExt;

type
  TLfmForm = class(TForm)
    NameLabel: TLabel;               { set by the .lfm, by name }
    NameEdit: TEdit;
    HelloButton: TButton;
    SizeGroup: TRadioGroup;
    StatusBar1: TStatusBar;
    procedure NameEditChange(Sender: TObject);   { found by name too }
  end;

var
  LfmForm: TLfmForm;

implementation

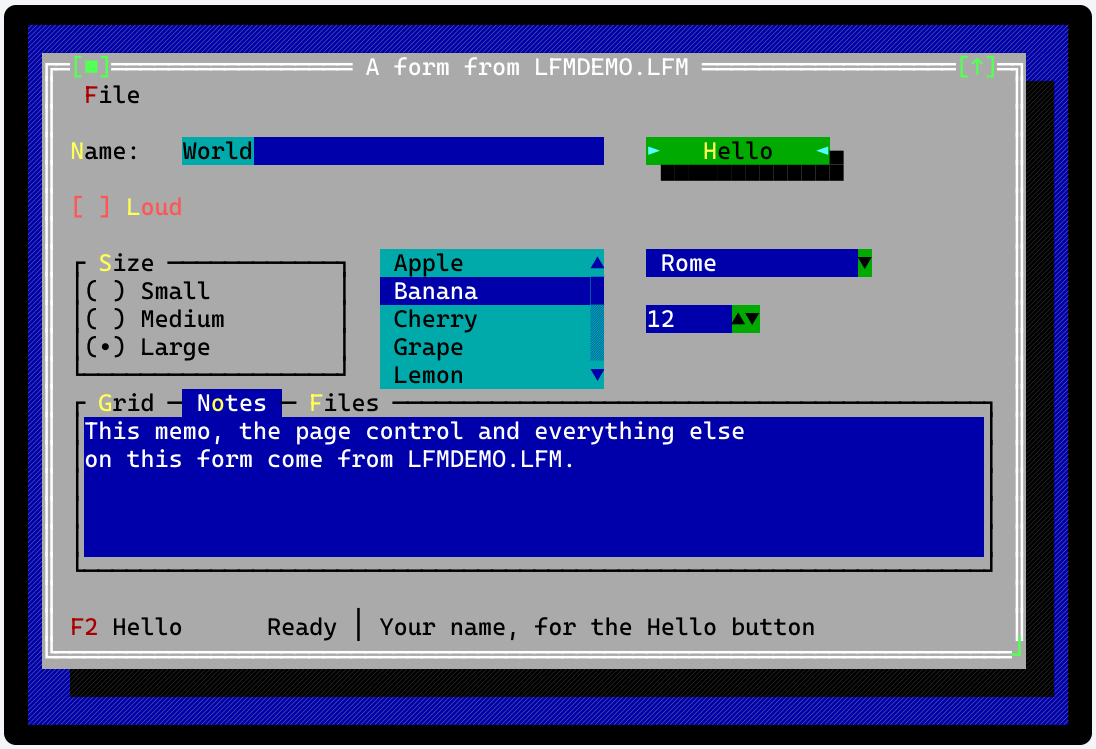
procedure TLfmForm.NameEditChange(Sender: TObject);
begin
  StatusBar1.Panels[1].Text := 'Name: ' + NameEdit.Text;
end;

end.
```

and `LFMDEMO.LFM` next to it says where everything is and how it is set up:

```
object LfmForm: TLfmForm
  Left = 3
  Top = 1
  Width = 70
  Height = 22
  Caption = 'A form from LFMDEMO.LFM'
  Position = poScreenCenter
  ActiveControl = NameEdit
  object NameLabel: TLabel
    Left = 1
    Top = 1
    Caption = '&Name:'
    FocusControl = NameEdit
  end
  object NameEdit: TEdit
    Left = 9
    Top = 1
    Width = 30
    Height = 1
    Anchors = [akTop, akLeft, akRight]
    Text = 'World'
    OnChange = NameEditChange
  end
  object SizeGroup: TRadioGroup
    Left = 1
    Top = 5
    Width = 20
    Height = 5
    Caption = '&Size'
    ItemIndex = 2
    Items.Strings = (
      'Small'
      'Medium'
      'Large'
    )
  end
  object StatusBar1: TStatusBar
    Panels = <
      item
        Text = '~F2~ Hello'
        Width = 12
      end
      item
        Text = 'Ready'
      end>
```

```
  end
end
```



The whole LfmDemo form: every control on it comes from LFMDEMO.LFM.

`Application.CreateForm(TLfmForm, LfmForm)` (or `TLfmForm.Create`) reads it. `TForm.Create` looks for the form's `.lfm` in this order:

1. text built into the program with `RegisterFormText`;
2. a file named with `RegisterFormFile(TLfmForm, 'FORMS\MAIN.LFM')`;
3. `<unit name>.lfm`, the Lazarus convention (`LfmDemo` -> `LFMDEMO.LFM`), in the current folder or the program's folder.

With none found the form is empty, as before. A form class can still have a constructor: it runs after `inherited Create` has read the `.lfm`.

To build the `.lfm` into the program, so no file is needed next to it, use LFM2INC (below).

LFM2INC: building an .lfm into the program

LFM2INC turns an `.lfm` file into an include file (`.LFI`) that holds the same text as one Pascal string constant. The program then carries the form with it,

and `TForm.Create` finds it before it looks for any file.

Building the tool. It is a small console program that uses only `Classes` and `SysUtils`; build it once, for DOS or for the machine you build on:

```
fpc -Tgo32v2 -O2 lfm2inc.pas     (DOS: LFM2INC.EXE, needs a DPMI host)
fpc -O2 lfm2inc.pas              (Windows / Linux, if you build there)
```

Running it.

```
LFM2INC form.lfm [form.lfi]
```

Argument	Meaning
`form.lfm`	the `.lfm` to read
`form.lfi`	the include file to write; left out: the same name with `.LFI` (`ABOUTLFM.LFM` -> `ABOUTLFM.LFI`)

It prints the name it wrote and the number of lines, for example `ABOUTLFM.LFI: 29 lines`. Run without arguments it shows the usage line.

What it writes. One string expression, a line per `.lfm` line, quotes doubled, each line ending in `#10`:

```
{ ABOUTLFM.LFI - made by LFM2INC from ABOUTLFM.LFM }
'object AboutForm: TAboutForm'#10 +
'  Left = 0'#10 +
'  Top = 0'#10 +
'  Width = 46'#10 +
'  Caption = ''About'''#10 +
...
'end'#10
```

Using it. Register the text in the form unit's `initialization` section. The form class stays as it was: fields for the controls, methods for the events, no constructor:

```
unit AboutLfm;
```

```
{$mode objfpc}{$H+}

interface

uses
  Classes, SysUtils, TUICtrl, TUIStd;

type
  TAboutForm = class(TForm)
    InfoLabel: TLabel;
    OkButton: TButton;
  end;

implementation

initialization
  RegisterFormText(TAboutForm, {$I ABOUTLFM.LFI});
end.
```

```
procedure TMainForm.AboutClick(Sender: TObject);
var f: TAboutForm;
begin
  f := TAboutForm.Create(nil);     { reads the built-in text }
  try
    f.ShowModal;
  finally
    f.Free;
  end;
end;
```

When you change the form. Edit the `.lfm`, run LFM2INC again, then rebuild: the `.LFI` is a copy, the compiler does not look at the `.lfm`. A batch file keeps the steps together:

```
@echo off
rem MAKE.BAT - the .lfm files into include files, then the program
lfm2inc ABOUTLFM.LFM
lfm2inc OPTIONS.LFM
fpc -Tgo32v2 -O2 myprog.pas
```

File or built in?

	`.lfm` file next to the program	built in with LFM2INC
how	nothing to do: `<unit name>.lfm` is found, or `RegisterFormFile`	`RegisterFormText(TForm1, {$I FORM1.LFI})`
change a form	edit the `.lfm`, run the program again	edit, run LFM2INC, rebuild
ship	the `.EXE` and its `.lfm` files	the `.EXE` alone
good for	trying layouts quickly	a finished program

Both can be mixed. Built-in text wins over a file, so a finished program uses its built-in forms; leave out the `RegisterFormText` line (for example in a test build) and the `.lfm` files are read instead.

Notes:

- The `.LFI` is plain Pascal: an `.lfm` of any size works (the constant is joined from one literal per line).
- Characters outside ASCII (box drawing, accented letters) are copied as they are; keep the `.lfm` in the code page the program shows (CP437 on DOS).
- The `.lfm` is checked when the form is created, not by LFM2INC: a typing error shows up as an exception (or, for an unknown property, a line in `LfmWarnings`) when the program runs. `LFMTEST.PAS` shows how to test forms automatically.

What an `.lfm` can hold:

In the .lfm	Example
any published property	`Caption = 'Hello'`, `Default = True`, `MaxLength = 4`
enums and sets	`Align = alClient`, `Anchors = [akTop, akLeft, akRight]`, `Options = [goVertLine, goEditing]`
colours and modal results by name	`Font.Color = clRed`, `Color = clCyan`, `ModalResult = mrOK`
sub-properties	`Font.Color`, `Constraints.MinWidth`
lists of strings	`Items.Strings = ( 'a' 'b' )`, `Lines.Strings = (...)`
collections	`StatusBar.Panels = < item ... end >`, `ListView.Columns = < ... >`

In the .lfm	Example
references to other components	`FocusControl = NameEdit`, `Menu = MainMenu1`, `Action = actHello`, `ActivePage = NotesSheet`, `PopupMenu = EditMenu`
events: the form's published methods	`OnClick = HelloButtonClick`
nested objects	controls inside panels, group boxes and tab sheets; tab sheets inside a page control; menu items inside a menu or item; actions inside an action list; timers, dialogs and menus anywhere
shortcuts	numbers, as Lazarus writes them: `ShortCut = 113` is F2, `16465` is Ctrl+Q (`scCtrl` 16384 + `Ord('Q')`)

Not in an `.lfm` (set them in `OnCreate`): string grid cells and column widths, list view items, tree view nodes, check list box check marks.

A property the control does not have (`DesignTimePPI`, `ParentFont`...) is skipped and noted in `LfmWarnings`; with `LfmStrict := true` it is an error instead. `ReadFormText(Form, Text)` and `ReadFormFile(Form, Name)` read an `.lfm` into any existing form. `LFMTEST.PAS` checks all of this.

What every control has

```
Edit1 := TEdit.Create(Self);    { the owner frees it }
Edit1.Parent := Self;           { where it is drawn: a form, panel, group box, tab
Edit1.SetBounds(9, 1, 26, 1);
```

Property / event	What it does
`Left`, `Top`, `Width`, `Height`, `SetBounds`	position and size in cells
`Caption` / `Text`	`'&'` marks the access key: Alt+letter, or the letter alone when the focus is not in a text field
`Enabled`, `Visible`	greyed / hidden; hidden controls are left out of the Tab order
`Color`, `Font.Color`	background and text colour

Property / event	What it does
`Hint`, `ShowHint`	status bar text and the hint window, see Hints
`TabOrder`, `TabStop`	the Tab order (creation order by default)
`Align`	`alTop`, `alBottom`, `alLeft`, `alRight`, `alClient`: fills that side of the parent
`Anchors`	`[akLeft, akTop, akRight]` stretches with the parent, `[akRight, akBottom]` moves with it
`Constraints`	`MinWidth`, `MinHeight`, `MaxWidth`, `MaxHeight`
`PopupMenu`	opens on a right click or Shift+F10
`Action`	a `TAction` gives Caption, Hint, Enabled... and the click
`OnClick`, `OnDblClick`	mouse clicks (and Enter/Space on buttons, check boxes)
`OnEnter`, `OnExit`	the focus came / went
`OnKeyDown`, `OnKeyPress`	keys (`VK_` codes, characters)
`OnMouseDown`, `OnMouseUp`, `OnMouseMove`, `OnMouseWheel`	the mouse, positions relative to the control
`SetFocus`, `Focused`, `CanFocus`	the focus

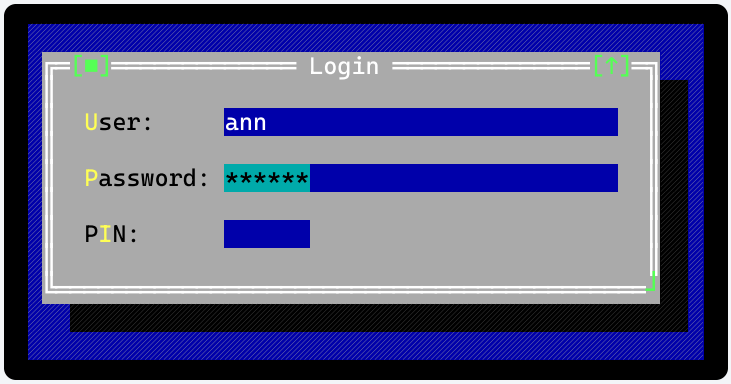
Labels and edit fields

```
UserEdit := TEdit.Create(Self);
UserEdit.Parent := Self;
UserEdit.SetBounds(12, 1, 28, 1);
UserEdit.Text := 'ann';
UserEdit.OnChange := @UserEditChange;

PassEdit := TEdit.Create(Self);
PassEdit.Parent := Self;
PassEdit.SetBounds(12, 3, 28, 1);
PassEdit.PasswordChar := '*';

PinEdit := TEdit.Create(Self);
PinEdit.Parent := Self;
PinEdit.SetBounds(12, 5, 6, 1);
PinEdit.MaxLength := 4;
```

```
UserLabel := TLabel.Create(Self);
UserLabel.Parent := Self;
UserLabel.Left := 2;
UserLabel.Top := 1;
UserLabel.Caption := '&User:';
UserLabel.FocusControl := UserEdit;     { Alt+U goes to UserEdit }
```



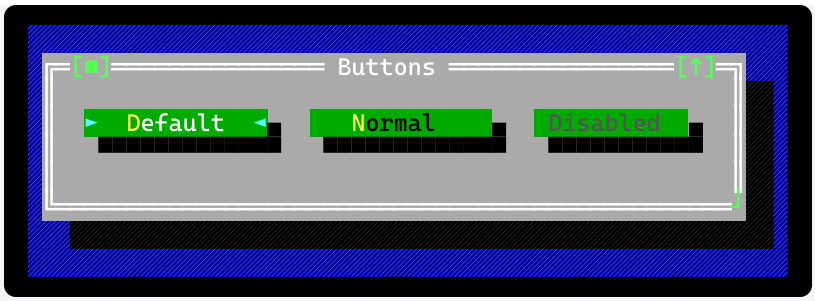
Labels with FocusControl; a password field has the focus.

`TLabel`: `Caption` (several lines with `LineEnding`), `Alignment`, `AutoSize` (on: the size follows the text), `FocusControl`, `ShowAccelChar`.

`TEdit`: `Text`, `MaxLength`, `PasswordChar`, `ReadOnly`, `AutoSelect` (entering with Tab selects the text, typing replaces it), `SelStart`, `SelectAll`, `Clear`, `CopyToClipboard`, `CutToClipboard`, `PasteFromClipboard`, `OnChange`. Keys: Home/End, Ctrl+Left/Right, Ins (insert / overwrite: a line or a block cursor), Ctrl+A, Ctrl+Y (clear), Ctrl+C/X/V.

Buttons

```
OkButton := TButton.Create(Self);
OkButton.Parent := Self;
OkButton.SetBounds(2, 1, 14, 2);
OkButton.Caption := '&Default';
OkButton.Default := true;        { Enter on the form clicks it; ►◄ marks it }
OkButton.OnClick := @OkButtonClick;
```



The default button (►◄), a normal one, a disabled one.

`Default` (Enter), `Cancel` (Esc), `ModalResult` (closes a modal form), `OnClick`. Space or Enter on a focused button clicks it; the mouse shows it pressed.

Check boxes, radio buttons, group boxes

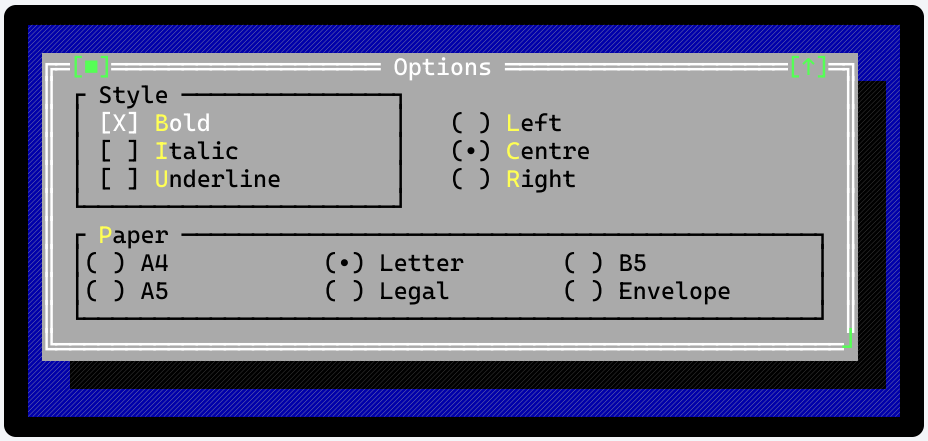
```
Style := TGroupBox.Create(Self);
Style.Parent := Self;
Style.SetBounds(1, 0, 24, 5);
Style.Caption := 'Style';

BoldCheck := TCheckBox.Create(Self);
BoldCheck.Parent := Style;           { inside the group box }
BoldCheck.Left := 1;
BoldCheck.Top := 0;
BoldCheck.Caption := '&Bold';
BoldCheck.Checked := true;
BoldCheck.OnChange := @StyleChange;

CentreRadio := TRadioButton.Create(Self);
CentreRadio.Parent := Self;
CentreRadio.Left := 28;
CentreRadio.Top := 2;
CentreRadio.Caption := '&Centre';
CentreRadio.Checked := true;         { the other radio buttons of the parent go of

Paper := TRadioGroup.Create(Self);
Paper.Parent := Self;
Paper.SetBounds(1, 5, 54, 4);
Paper.Caption := '&Paper';
Paper.Columns := 3;
Paper.Items.CommaText := 'A4,A5,Letter,Legal,B5,Envelope';
```

```
Paper.ItemIndex := 2;
Paper.OnClick := @PaperClick;
```



A group box with check boxes, three radio buttons, a radio group with three columns.

- `TCheckBox`: `Checked`, `OnChange`, `OnClick`; Space toggles.
- `TRadioButton`: radio buttons with the same parent belong together.
- `TGroupBox`: a frame with a caption; its children sit inside.
- `TRadioGroup` (`TUIEXT`): `Items`, `ItemIndex`, `Columns`, `OnClick`, `OnSelectionChanged`. The arrows move inside the group; only the checked button is a Tab stop.

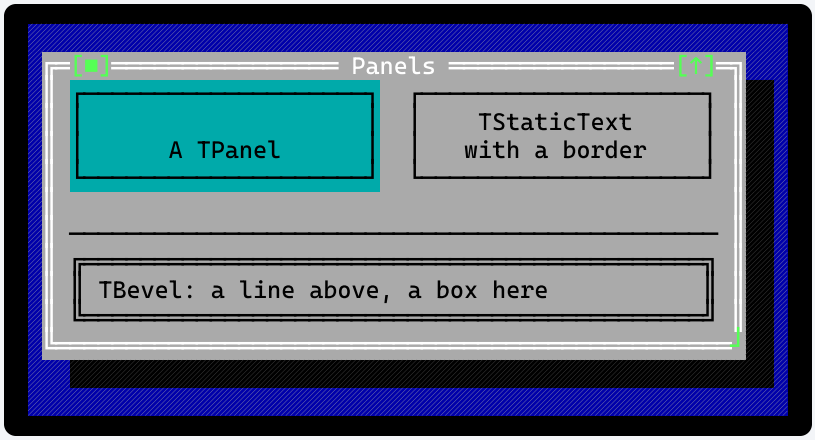
Panels, bevels, static text

```
P := TPanel.Create(Self);
P.Parent := Self;
P.SetBounds(1, 0, 22, 4);
P.Border := pbSingle;            { pbNone, pbSingle, pbDouble }
P.Caption := 'A TPanel';         { shown while the panel has no children }
P.Color := clCyan;

S := TStaticText.Create(Self);
S.Parent := Self;
S.SetBounds(25, 0, 22, 4);
S.BorderStyle := sbsSingle;      { sbsNone, sbsSingle, sbsSunken }
S.Alignment := taCenter;
S.Caption := 'TStaticText' + LineEnding + 'with a border';

B := TBevel.Create(Self);
```

```
B.Parent := Self;
B.SetBounds(1, 5, 46, 1);
B.Shape := bsTopLine;              { bsBox, bsFrame, bsTopLine, bsBottomLine, bsLeft
```



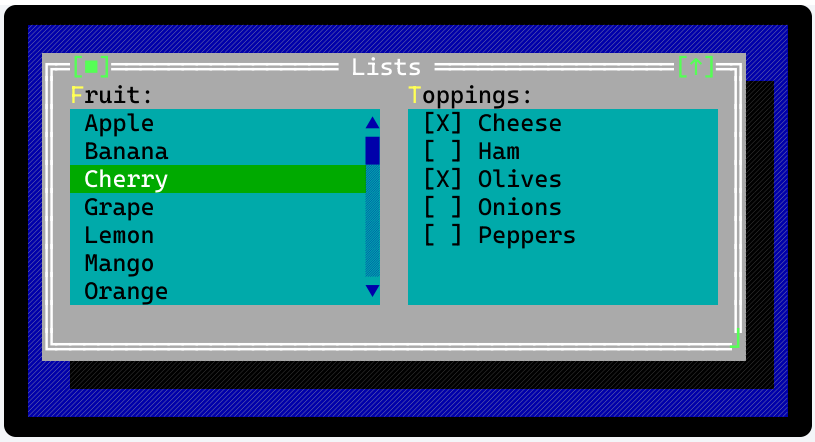
A TPanel, a TStaticText with a border, TBevel lines and a box.

`TBevel.Style`: `bsLowered` draws single lines, `bsRaised` double lines. Panels are containers: give them children, `Align` and `Anchors`.

List boxes

```
FruitList := TListBox.Create(Self);
FruitList.Parent := Self;
FruitList.SetBounds(1, 1, 22, 7);
FruitList.Items.CommaText := 'Apple,Banana,Cherry,Grape,Lemon,Mango,Orange,Peach,P
FruitList.ItemIndex := 2;
FruitList.OnSelectionChange := @FruitListSelectionChange;
FruitList.OnDblClick := @FruitListDblClick;

Toppings := TCheckListBox.Create(Self);
Toppings.Parent := Self;
Toppings.SetBounds(25, 1, 22, 7);
Toppings.Items.CommaText := 'Cheese,Ham,Olives,Onions,Peppers';
Toppings.Checked[0] := true;
Toppings.OnClickCheck := @ToppingsClickCheck;
```



A list box with a scroll bar, and a check list box.

```
procedure TMyForm.FruitListSelectionChange(Sender: TObject; User: boolean);
begin
  if FruitList.ItemIndex >= 0 then
    StatusLabel.Caption := FruitList.Items[FruitList.ItemIndex];
end;
```

`TListBox`: `Items` (a `TStrings`: `Add`, `Delete`, `IndexOf`, `CommaText`...), `ItemIndex`, `TopIndex`, `Sorted`, `Count`, `Clear`, `Frame`, `ItemAtPos`, `OnSelectionChange`, `OnClick`, `OnDblClick`. The scroll bar shows when the items do not fit. Keys: arrows, PgUp/PgDn, Home/End, a letter jumps to the next item starting with it.

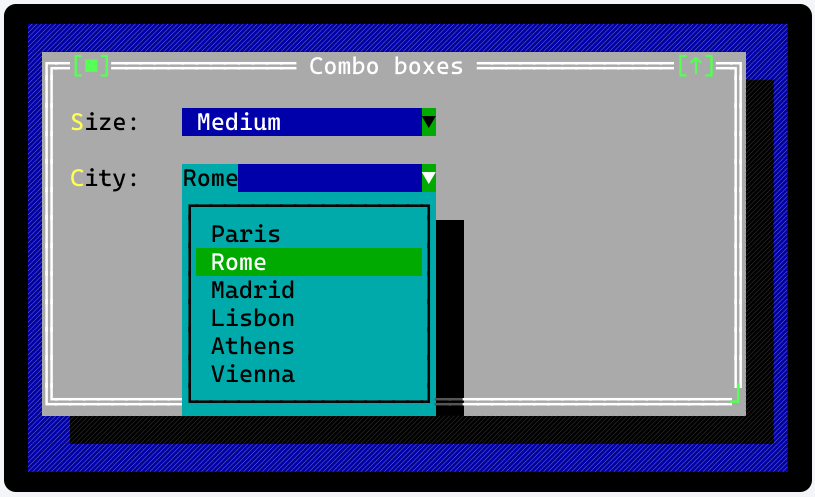
`TCheckListBox` (`TUIEXT`): a list box with `Checked[]`, `CheckAll`, `Toggle`, `OnClickCheck`; Space or a click on `[ ]` checks.

Combo boxes

```
SizeCombo := TComboBox.Create(Self);
SizeCombo.Parent := Self;
SizeCombo.SetBounds(9, 1, 18, 1);
SizeCombo.Style := csDropDownList;     { pick only }
SizeCombo.Items.CommaText := 'Small,Medium,Large,"Extra large"';
SizeCombo.ItemIndex := 1;
SizeCombo.OnSelect := @SizeComboSelect;

CityCombo := TComboBox.Create(Self);
```

```
CityCombo.Parent := Self;
CityCombo.SetBounds(9, 3, 18, 1);      { csDropDown (default): type or pick }
CityCombo.Items.CommaText := 'Paris,Rome,Madrid,Lisbon,Athens,Vienna';
CityCombo.Text := 'Rome';
```

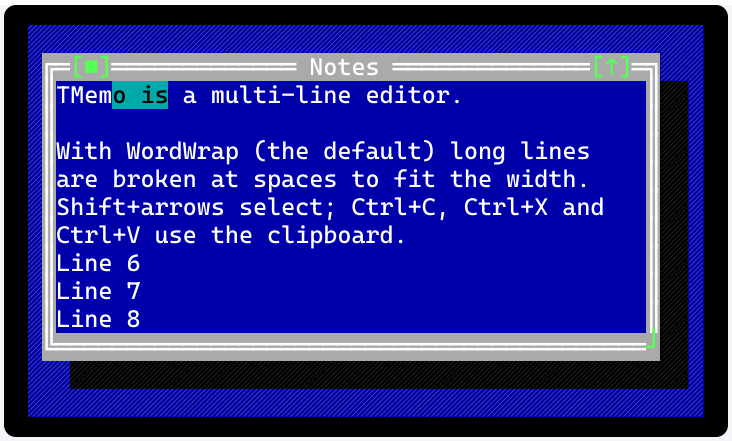


A pick-only combo box, and an editable one with its list open.

`Style` (`csDropDown`, `csDropDownList`), `Items`, `ItemIndex`, `Text`, `Sorted`, `DropDownCount`, `DroppedDown`, `DropDown`, `CloseUp`; `OnChange` (the text changed), `OnSelect` (an item was picked), `OnDropDown`, `OnCloseUp`. Up/Down change the item; Alt+Down or F4 (or a click on ▼) opens the list; Enter picks, Esc closes.

Memo

```
Memo1 := TMemo.Create(Self);
Memo1.Parent := Self;
Memo1.Align := alClient;
Memo1.ScrollBars := ssAutoVertical;
Memo1.Lines.Add('TMemo is a multi-line editor.');
Memo1.Lines.Add('');
Memo1.Lines.Add('With WordWrap (the default) long lines are broken at spaces to fi
Memo1.SelStart := 4;          { select "is a" }
Memo1.SelLength := 4;
```



A memo with Align alClient, word wrap and a selection.

`Lines` (`TStrings`: `LoadFromFile`, `SaveToFile`...), `Text`, `WordWrap`, `ScrollBars` (`ssNone`, `ssHorizontal`, `ssVertical`, `ssBoth`, `ssAutoHorizontal`, `ssAutoVertical`, `ssAutoBoth`), `ReadOnly`, `WantReturns`, `WantTabs`, `CaretPos`, `SelStart`, `SelLength`, `SelText`, `SelectAll`, `ClearSelection`, `CopyToClipboard`, `CutToClipboard`, `PasteFromClipboard`, `Clear`, `Append`, `OnChange`.

Keys: arrows (screen rows when wrapped), Home/End, Ctrl+Home/End, PgUp/PgDn, Ctrl+Left/Right, Shift+ any of them selects, Ctrl+Y deletes the line, Ctrl+A, Ctrl+C/X/V. Mouse: click, drag, double click (a word), the scroll bars. `MemoFindNext` and `MemoReplace` (`TUIDLGS`) search it.

Numbers and ranges

```
Qty := TSpinEdit.Create(Self);        { TUIEXT }
Qty.Parent := Self;
Qty.SetBounds(12, 1, 8, 1);
Qty.MinValue := 1;
Qty.MaxValue := 99;
Qty.Value := 12;

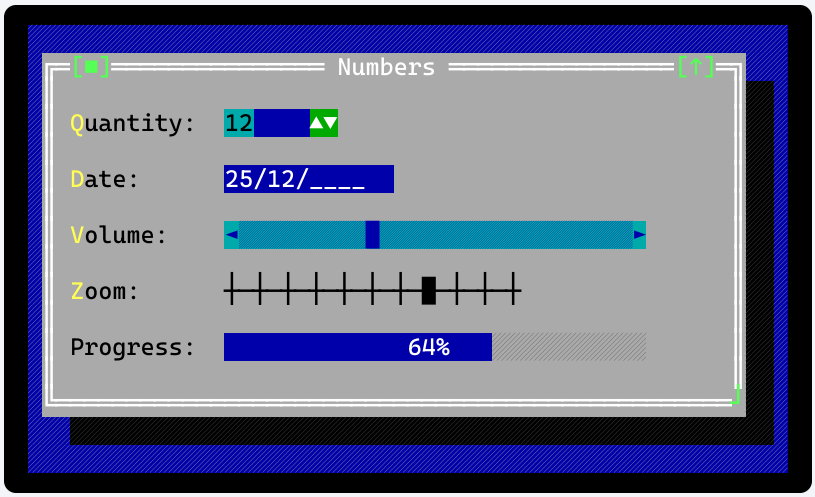
DateEdit := TMaskEdit.Create(Self);
DateEdit.Parent := Self;
DateEdit.SetBounds(12, 3, 12, 1);
DateEdit.EditMask := '99/99/9999;1;_';
DateEdit.Text := '2512';              { only the digits: 25/12/____ }

Volume := TScrollBar.Create(Self);
```

```
Volume.Parent := Self;
Volume.SetBounds(12, 5, 30, 1);
Volume.Position := 35;                  { Min 0, Max 100 by default }

Zoom := TTrackBar.Create(Self);         { TUIVIEWS }
Zoom.Parent := Self;
Zoom.SetBounds(12, 7, 21, 1);
Zoom.Max := 10;
Zoom.Position := 7;

Bar := TProgressBar.Create(Self);
Bar.Parent := Self;
Bar.SetBounds(12, 9, 30, 1);
Bar.Position := 64;
Bar.BarShowText := true;
```



TSpinEdit, TMaskEdit, TScrollBar, TTrackBar and TProgressBar.

- `TSpinEdit`: `Value`, `MinValue`, `MaxValue`, `Increment`; Up/Down, PgUp/PgDn (ten steps), the `▲▼` arrows; only digits can be typed.
- `TMaskEdit`: `EditMask` is `mask;save literals;blank`. Mask characters: `0` digit, `9` digit or blank, `#` digit/sign, `L` / `l` letter, `A` / `a` letter or digit, `C` / `c` any (capital = required), `>` `<` upper/lower case, `\x` the literal x. `Text` (literals only with save literals 1), `EditText`, `ValidateEdit` (all required positions filled).
- `TScrollBar`: `Kind` (`sbHorizontal`, `sbVertical`), `Min`, `Max`, `Position`, `SmallChange`, `LargeChange`, `OnChange`.
- `TTrackBar`: `Min`, `Max`, `Position`, `Orientation`, `Frequency` (ticks), `LineSize`, `PageSize`, `OnChange`.

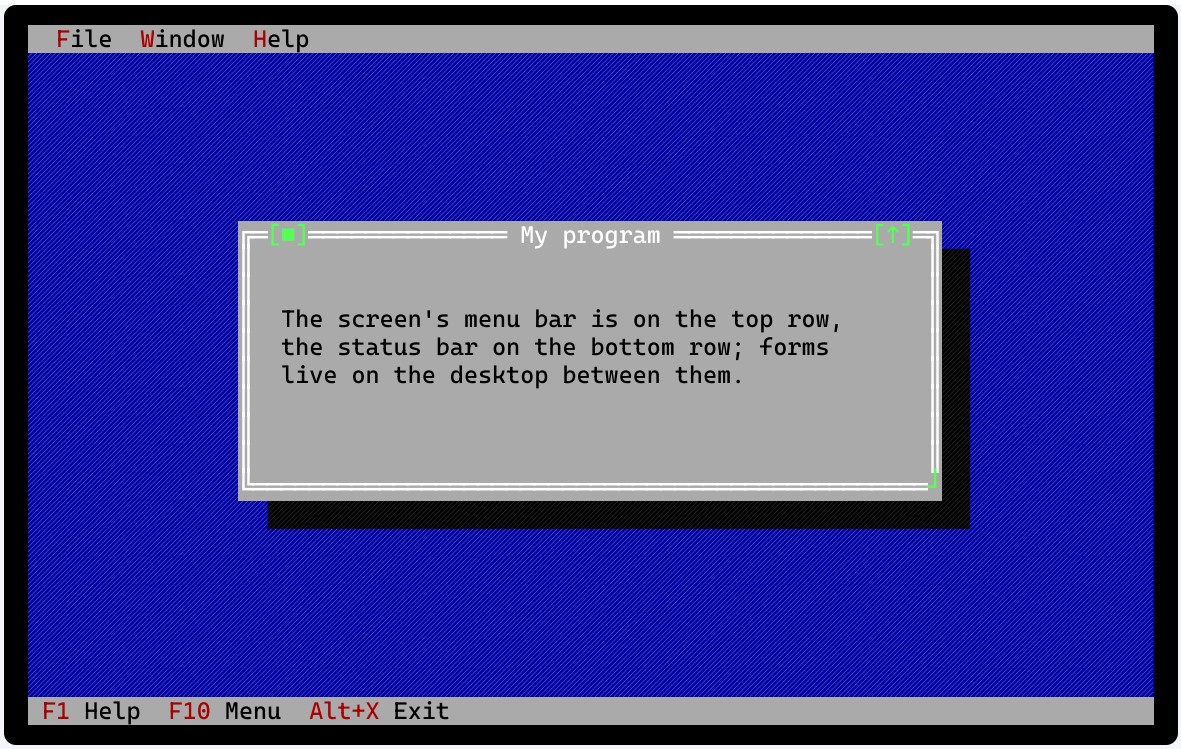
- `TProgressBar`: `Min`, `Max`, `Position`, `Step`, `StepIt`, `StepBy`, `Orientation`, `BarShowText`.

Menus and status bars

Menus and status bars work at two levels. On the **screen** the menu bar takes the top row and the status bar the bottom row; forms live on the desktop between them:

```
ScreenMenu := TMainMenu.Create(Self);
ScreenMenu.Items.Add(NewSubMenu('&File', 0, '', [
  NewItem('E&xit', ShortCut(Ord('X'), [ssAlt]), false, true, @ExitClick, 0, '')]))
ScreenMenu.Items.Add(NewSubMenu('&Window', 0, '', [
  NewItem('&Tile', 0, false, true, @TileClick, 0, '')]));
ScreenMenu.Items.Add(NewSubMenu('&Help', 0, '', [
  NewItem('&About', 0, false, true, @AboutClick, 0, '')]));
Application.Menu := ScreenMenu;

ScreenStatus := TStatusBar.Create(Self);   { no Parent: the screen's }
ScreenStatus.Panels.Add.Text := '~F1~ Help';
ScreenStatus.Panels.Add.Text := '~F10~ Menu';
ScreenStatus.Panels.Add.Text := '~Alt+X~ Exit';
Application.StatusBar := ScreenStatus;
```

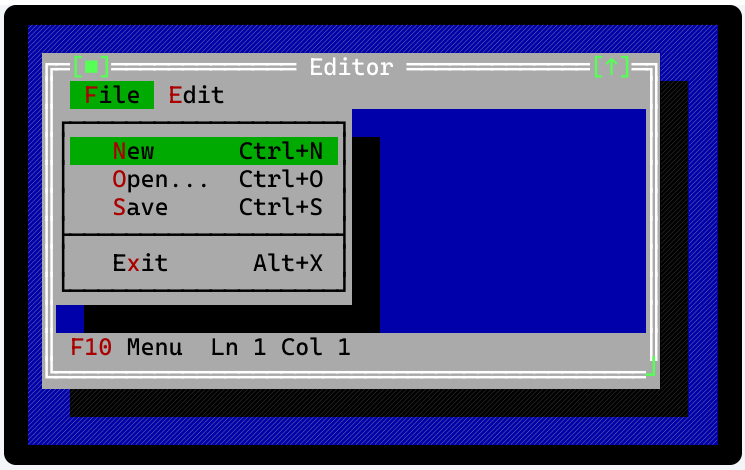


The whole screen: the menu bar, the desktop with a form, the status bar.

In a **window**, `Menu` puts the bar under the title, and a status bar with the form as `Parent` takes the last row:

```
FileMenu := TMenuItem.Create(Self);         { item by item, as a .lfm does }
FileMenu.Caption := '&File';
Menu1 := TMainMenu.Create(Self);
Menu1.Items.Add(FileMenu);
NewItem1 := TMenuItem.Create(Self);
NewItem1.Caption := '&New';
NewItem1.ShortCut := ShortCut(Ord('N'), [ssCtrl]);
NewItem1.OnClick := @NewClick;
FileMenu.Add(NewItem1);
FileMenu.Add(NewLine);
Menu := Menu1;

Status := TStatusBar.Create(Self);
Status.Parent := Self;                      { Align alBottom }
Status.Panels.Add.Text := '~F10~ Menu';
Status.Panels.Add.Text := 'Ln 1 Col 1';
```



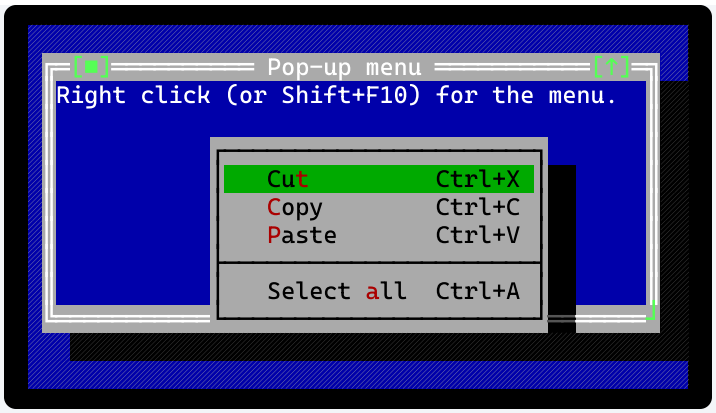
A window menu bar (File open) and a window status bar.

`TMenuItem`: `Caption` (`'-'` is a line), `ShortCut` (`ShortCut(VK_F3, [])`, `TextToShortCut('Ctrl+N')`), `Hint`, `Enabled`, `Visible`, `Checked`, `AutoCheck`, `OnClick`, `Action`, `Add`, `Insert`, `Delete`, `Clear`, `Count`, `Items[]`. An item with sub items gets `OnClick` when they open, so it can fill them (a list of the open windows).

Keys: F10 opens the active form's menu bar, else the screen's. Alt+letter tries the form's menu bar, then the form's controls, then the screen's menu bar. Shortcuts work whenever their form is active.

A **pop-up menu** opens where it is needed:

```
EditMenu := TPopupMenu.Create(Self);
EditMenu.Items.Add([
  NewItem('Cu&t', ShortCut(Ord('X'), [ssCtrl]), false, true, @CutClick, 0, ''),
  NewItem('&Copy', ShortCut(Ord('C'), [ssCtrl]), false, true, @CopyClick, 0, ''),
  NewItem('&Paste', ShortCut(Ord('V'), [ssCtrl]), false, true, @PasteClick, 0, '')
  NewLine,
  NewItem('Select &all', ShortCut(Ord('A'), [ssCtrl]), false, true, @SelectAllClic
Memo1.PopupMenu := EditMenu;          { right click or Shift+F10 }
```



A pop-up menu on a memo.

`TPopupMenu`: `Popup(X, Y)`, `OnPopup`, `OnClose`, `PopupComponent` (the control it opened for).

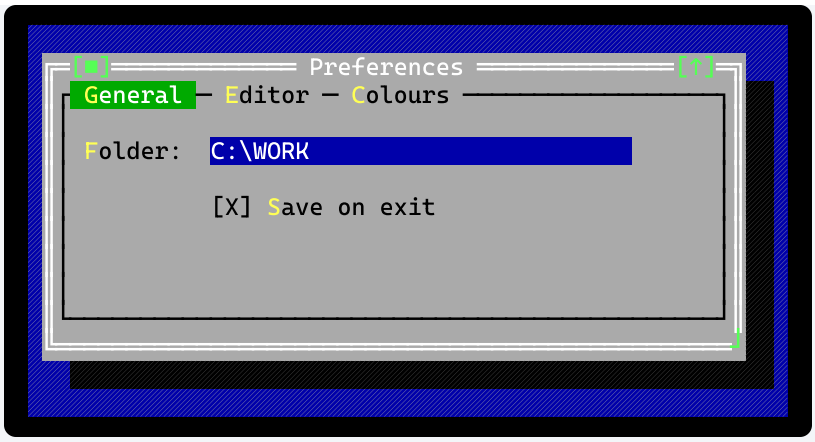
`TStatusBar`: `Panels` (`Add`; each `TStatusPanel` has `Text`, `Width` (0 = fit the text), `Alignment`, and the TUI extras `OnClick` and `ShortCut`), `SimplePanel` and `SimpleText`, `AutoHint`. In panel text `'~'` switches the highlight on and off.

Page control

```
Pages := TPageControl.Create(Self);
Pages.Parent := Self;
Pages.Align := alClient;

General := Pages.AddTabSheet;
General.Caption := '&General';
Editor := Pages.AddTabSheet;
Editor.Caption := '&Editor';

FolderEdit := TEdit.Create(Self);
FolderEdit.Parent := General;          { on the General page }
FolderEdit.SetBounds(10, 1, 30, 1);
```



A page control with three tab sheets.

`Pages[]`, `PageCount`, `ActivePage`, `ActivePageIndex`, `AddTabSheet`, `SelectNextPage`, `OnChange`, `OnChanging` (`AllowChange := false` refuses). `TTabSheet`: `Caption`, `PageControl`, `PageIndex`, `TabVisible`.

Ctrl+Tab, Ctrl+Shift+Tab and Ctrl+PgDn/PgUp switch pages from inside; on the tab row Left/Right; Alt+letter picks a tab; a click on a tab. Controls on hidden pages are out of the Tab order.

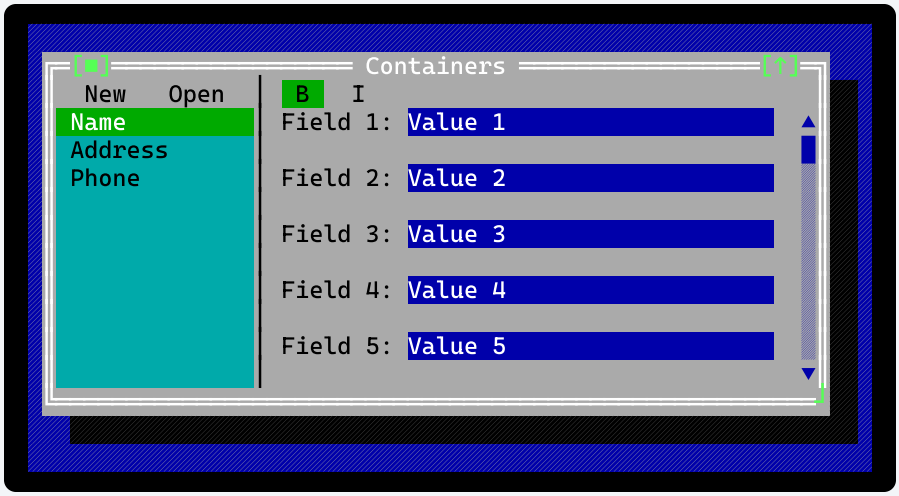
Tool bar, splitter, scroll box

```
Tools := TToolBar.Create(Self);          { Align alTop }
Tools.Parent := Self;
NewBtn := TToolButton.Create(Self);
NewBtn.Parent := Tools;                  { laid out left to right }
NewBtn.Caption := 'New';
NewBtn.OnClick := @NewClick;
Sep := TToolButton.Create(Self);
Sep.Parent := Tools;
Sep.Style := tbsSeparator;
BoldBtn := TToolButton.Create(Self);
BoldBtn.Parent := Tools;
BoldBtn.Style := tbsCheck;               { stays down }
BoldBtn.Caption := 'B';

Sections := TListBox.Create(Self);
Sections.Parent := Self;
Sections.Width := 14;
Sections.Align := alLeft;
```

```
Splitter1 := TSplitter.Create(Self);    { after the control it resizes }
Splitter1.Parent := Self;
Splitter1.Align := alLeft;

Box := TScrollBox.Create(Self);
Box.Parent := Self;
Box.Align := alClient;                  { put more controls in it than fit }
```



A tool bar, a list (alLeft), a splitter and a scroll box (alClient).

- `TToolBar` / `TToolButton` (`TUIPAGES`): `Style` (`tbsButton`, `tbsCheck`, `tbsSeparator`), `Down`, `GroupIndex` (one down per group), `AllowAllUp`. Tool buttons never take the focus.
- `TSpeedButton`: the same button without a tool bar.
- `TSplitter`: drag it; `MinSize`, `OnMoved`.
- `TScrollBox`: scroll bars when the children do not fit; the focused control is scrolled into view; PgUp/PgDn; `ScrollBy`, `ScrollInView`, `Border`.

String grid

```
Grid := TStringGrid.Create(Self);       { TUIGRID }
Grid.Parent := Self;
Grid.Align := alClient;
Grid.ColCount := 4;
Grid.RowCount := 7;
Grid.FixedCols := 0;                    { FixedRows stays 1: the titles }
```

```
Grid.Options := Grid.Options + [goEditing, goColSizing];
Grid.ColWidths[0] := 16;                { a column's last cell is its line }
Grid.ColAlignments[1] := taRightJustify;
Grid.Cells[0, 0] := 'Product';
Grid.Cells[1, 0] := 'Qty';
Grid.Cells[0, 1] := 'Apples';
Grid.Cells[1, 1] := '2';
Grid.OnHeaderClick := @GridHeaderClick;     { sort here }
Grid.OnSetEditText := @GridSetEditText;     { a cell was edited }
```



A string grid with a title row and right aligned numbers.

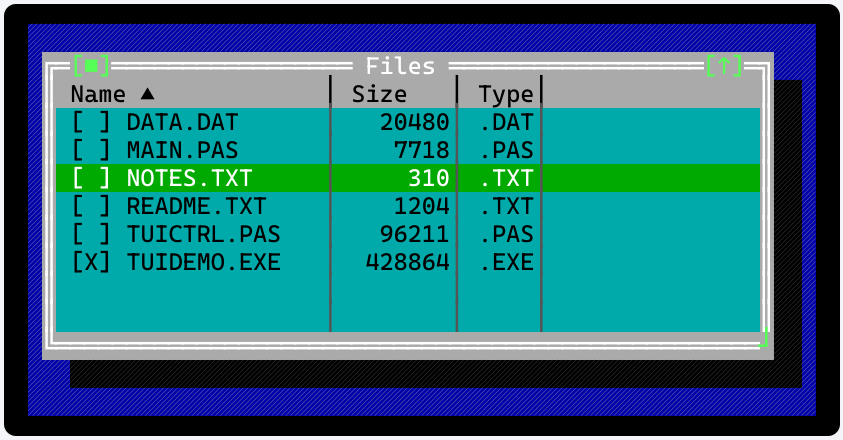
```
procedure TMyForm.GridHeaderClick(Sender: TObject; IsColumn: boolean; Index: integ
begin
  if IsColumn then Grid.SortColRow(true, Index);   { numbers sort as numbers }
end;
```

`ColCount`, `RowCount`, `FixedCols`, `FixedRows`, `ColWidths[]`, `DefaultColWidth`, `ColAlignments[]`, `Col`, `Row`, `TopRow`, `LeftCol`, `Options` (`goVertLine`, `goFixedVertLine`, `goColSizing`, `goEditing`, `goTabs`, `goRowSelect`), `Cells[]`, `SortColRow` + `SortOrder`, `DeleteColRow`, `Clean`, `MouseToCell`, `CellRect`, `EditorMode`. Events: `OnSelectCell` (`CanSelect := false` refuses), `OnSelection`, `OnDrawCell`, `OnHeaderClick`, `OnGetEditText`, `OnSetEditText`. `TDrawGrid` draws nothing itself: `OnDrawCell` paints with `Canvas`.

Editing (`goEditing`): F2 or Enter, or just type. In the editor Enter keeps the text, Esc throws it away, Up/Down keep it and move.

List view

```
Files := TListView.Create(Self);        { TUIVIEWS }
Files.Parent := Self;
Files.Align := alClient;
Files.Checkboxes := true;
Files.GridLines := true;
with Files.Columns.Add do begin Caption := 'Name'; Width := 20; end;
with Files.Columns.Add do begin Caption := 'Size'; Width := 9; Alignment := taRigh
with Files.Columns.Add do begin Caption := 'Type'; Width := 6; end;
with Files.Items.Add do
begin
  Caption := 'README.TXT';
  SubItems.Add('1204');
  SubItems.Add('.TXT');
end;
Files.OnSelectItem := @FilesSelectItem;
```

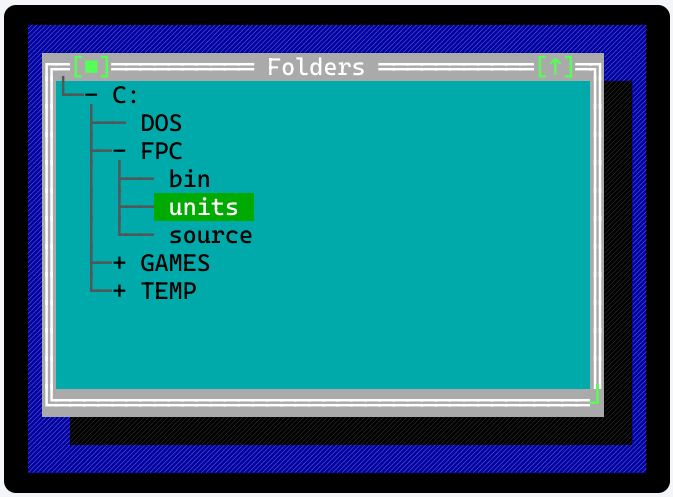


A list view in report style, sorted by Name, with check boxes and grid lines.

`ViewStyle` (`vsReport` with the header, `vsList`), `Columns` (`Caption`, `Width`, `Alignment`), `Items` (`Add`, `Insert`, `Delete`, `Clear`, `FindCaption`; each `TListItem` has `Caption`, `SubItems`, `Checked`, `Data`, `Selected`, `Index`), `ItemIndex`, `Selected`, `Checkboxes`, `GridLines`, `ShowColumnHeaders`, `AutoSort` (a click on a header sorts by it, again the other way), `SortColumn`, `SortDirection`, `AlphaSort`, `OnCompare`, `OnSelectItem`, `OnColumnClick`, `OnItemChecked`, `OnDblClick`.

Tree view

```
Tree := TTreeView.Create(Self);
Tree.Parent := Self;
Tree.Align := alClient;
Root := Tree.Items.Add(nil, 'C:');
Tree.Items.AddChild(Root, 'DOS');
Fpc := Tree.Items.AddChild(Root, 'FPC');
Tree.Items.AddChild(Fpc, 'bin');
Tree.Items.AddChild(Fpc, 'units');
Temp := Tree.Items.AddChild(Root, 'TEMP');
Temp.HasChildren := true;               { filled in OnExpanding }
Root.Expanded := true;
Tree.OnExpanding := @TreeExpanding;
Tree.OnChange := @TreeChange;
```



A tree view: C: and FPC open, GAMES and TEMP closed.

```
procedure TMyForm.TreeExpanding(Sender: TObject; Node: TTreeNode; var AllowExpansi
begin
  if (Node.Text = 'TEMP') and (Node.Count = 0) then
    Tree.Items.AddChild(Node, 'FILE1.TMP');          { the children, when needed }
end;

procedure TMyForm.TreeChange(Sender: TObject; Node: TTreeNode);
begin
  PathLabel.Caption := Node.GetTextPath;              { 'C:/FPC/units' }
end;
```

`Items` (`Add`, `AddFirst`, `AddChild`, `AddChildFirst`, `Insert`, `Delete`, `Clear`, `BeginUpdate`/`EndUpdate`, `GetFirstNode`, `FindNodeWithText`, `Count`, `Item[]`), `Selected`, `ShowLines`, `ShowRoot`, `ShowButtons`, `FullExpand`, `FullCollapse`. A `TTreeNode` has `Text`, `Data`, `Expanded`, `HasChildren`, `Count`, `Items[]`, `Parent`, `Level`, `Index`, `Expand`/`Collapse(Recurse)`, `Delete`, `GetFirstChild`, `GetNextSibling`, `GetNext`, `MakeVisible`, `GetTextPath`. Events: `OnChange`, `OnChanging`, `OnExpanding`, `OnExpanded`, `OnCollapsing`, `OnCollapsed`.

Keys: Right opens a node (or goes to its first child), Left closes it (or goes to its parent), `+` `-` `*`, a letter jumps. A click on `+`/`-` opens or closes it; a double click too.

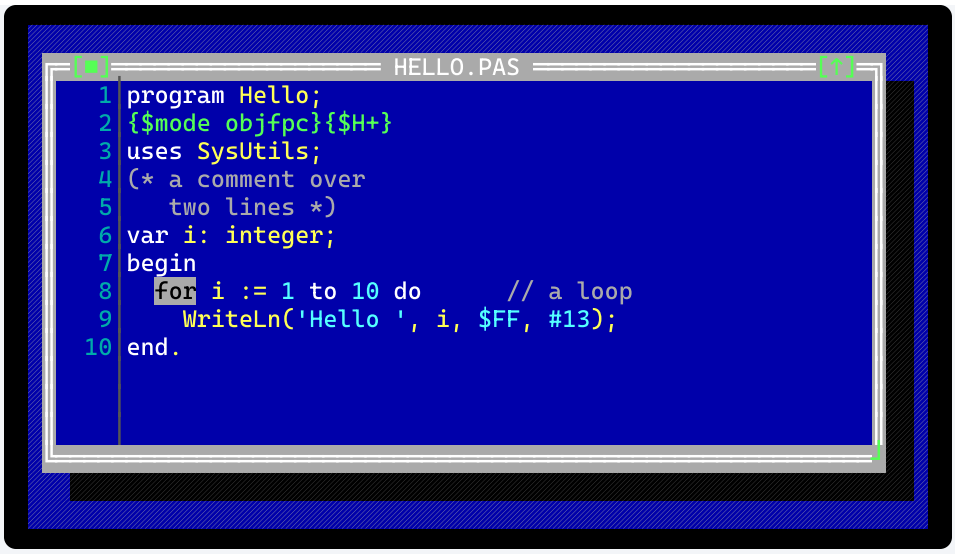
Code editor

`TSynEdit` (`TUISYNED`) is an editor for program text, with syntax colouring from a highlighter (`TUISYN`). The names follow Lazarus's SynEdit.

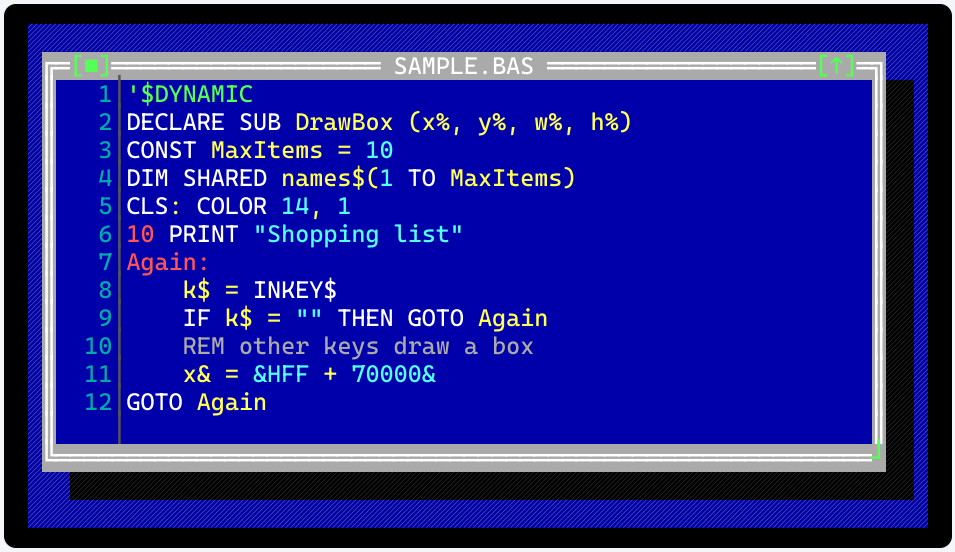
```
PasSyn := TSynPasSyn.Create(Self);       { Pascal }
BasSyn := TSynQBasicSyn.Create(Self);    { QBasic / QuickBASIC }

Editor := TSynEdit.Create(Self);
Editor.Parent := Self;
Editor.Align := alClient;
Editor.Highlighter := PasSyn;
Editor.Lines.LoadFromFile('HELLO.PAS');
Editor.OnStatusChange := @EditorStatusChange;   { the caret moved... }

procedure TMyForm.EditorStatusChange(Sender: TObject);
begin
  Status.Panels[0].Text := 'Ln ' + IntToStr(Editor.CaretY) + ' Col ' + IntToStr(Ed
end;
```



TSynEdit with the Pascal highlighter: keywords, a directive, a comment over two lines, strings, numbers; a selection.



TSynEdit with the QBasic highlighter: a metacommand, labels and a line number, strings, REM, &H numbers.

The editor. No word wrap (long lines scroll sideways), a gutter with line numbers, both scroll bars. As in the Turbo Pascal and QBasic editors the caret may stand past the end of a line; typing there fills the gap with spaces. Tabs in the text become spaces (`TabWidth` , default 8).

`Lines` , `Text` , `Highlighter` , `ReadOnly` , `Modified` , `InsertMode` , `TabWidth` , `BlockIndent` , `WordStarKeys` , `Options` (`eoScrollPastEol` , `eoTabsToSpaces` , `eoKeepCaretX` , `eoAutoIndent` , `eoBackspaceUnindents`), `Gutter` (`Visible` , `ShowLineNumbers` , `Color` , `TextColor`), `SelectedColor` , `Color` / `Font.Color` (the text's own colours), `CaretX` , `CaretY` , `CaretXY` (1 based), `TopLine` ,

`LeftChar`, `LinesInWindow`, `CharsInWindow`, `BlockBegin`, `BlockEnd`, `SelAvail`, `SelText`, `SelectAll`, `SelectWord`, `SelectLine`, `ClearSelection`, `CopyToClipboard`, `CutToClipboard`, `PasteFromClipboard`, `ClearAll`, `LoadFromFile`, `SaveToFile`, `BeginUpdate` / `EndUpdate`; `OnChange`, `OnStatusChange` (caret, selection, `Modified`, `InsertMode`).

Keys: arrows, Home (the first non-blank, again: column 1), End, Ctrl+Home/End, PgUp/PgDn, Ctrl+Left/Right (words), Ctrl+Up/Down (scroll), Shift+ any of them selects; Enter, Tab, Backspace, Del, Ins, Ctrl+Y (delete the line), Ctrl+A, Ctrl+C/X/V, Ctrl+Ins, Shift+Del, Shift+Ins, and the Turbo / WordStar keys below. Mouse: click, drag, double click (a word), a click in the gutter (the line), the scroll bars, the wheel.

Undo and redo. Every action is one undo step: an Enter with its indent, a paste over a selection, a block indent, a Ctrl+K V move. Typing goes back a word at a time. Undo puts the caret and the selection back where they were; undoing to the last save makes the text unmodified again (`Modified := false` marks that point, `SaveToFile` does it too). Text set from code (`Text`, `Lines`, `LoadFromFile`) clears the history.

Key	Does
Ctrl+Z, Alt+Backspace	undo
Ctrl+Shift+Z, Alt+Shift+Backspace	redo

`Undo`, `Redo`, `CanUndo`, `CanRedo`, `UndoCount`, `ClearUndo`, `MaxUndo` (1024 changes; the oldest steps go first), and `BeginUndoBlock` / `EndUndoBlock` to make several edits from code one step.

Indenting. With `eoAutoIndent` (on) Enter indents the new line like the line above. With `eoBackspaceUnindents` (on) Backspace in the leading blanks goes back to the indent of the nearest line above with a smaller one. Tab on a selection moves its lines right by `BlockIndent` (2), Shift+Tab moves them (or the caret's line) left; the selection stays. `IndentBlock` and `UnindentBlock` do it from code.

Turbo blocks and quick moves. Ctrl+K and Ctrl+Q wait for a second key (with or without Ctrl); `PendingPrefix` says which (the code editor window shows `^K`

in its status bar). A block marked with Ctrl+K B and Ctrl+K K stays while the caret moves, as in Turbo Pascal; the block commands use the selection when there is one.

Keys	Does
Ctrl+K B, Ctrl+K K	mark the start / the end of the block
Ctrl+K T, Ctrl+K L	mark the word / the line
Ctrl+K C, Ctrl+K V	copy / move the block to the caret
Ctrl+K Y	delete the block
Ctrl+K I, Ctrl+K U	indent / unindent the block
Ctrl+K H	unmark the block
Ctrl+Q S, Ctrl+Q D	the start / the end of the line
Ctrl+Q E, Ctrl+Q X	the top / the bottom of the window
Ctrl+Q R, Ctrl+Q C	the start / the end of the file
Ctrl+Q B, Ctrl+Q K	the start / the end of the block
Ctrl+Q Y	delete to the end of the line

`BlockMarked` and `MarkedText` tell about the marked block.

WordStar keys. `WordStarKeys := true` adds the single Ctrl keys of WordStar and the Turbo Pascal editor. They take the place of the CUA Ctrl+A, C, V, X and Z; use Ctrl+Ins, Shift+Ins, Shift+Del and Alt+Backspace for those then.

Key	Does	Key	Does
Ctrl+E	up	Ctrl+X	down
Ctrl+S	left	Ctrl+D	right
Ctrl+A	the word before	Ctrl+F	the next word
Ctrl+R	page up	Ctrl+C	page down
Ctrl+W	scroll up	Ctrl+Z	scroll down
Ctrl+G	delete the character	Ctrl+T	delete the word
Ctrl+Y	delete the line	Ctrl+V	insert / overwrite

Key	Does	Key	Does
Ctrl+N	insert a line		

The highlighters. One highlighter can serve several editors.

Highlighter	Knows
`TSynPasSyn`	TP / FPC / Delphi keywords; `{ }` `(* *)` `//` comments (the first two over several lines); `{$ }` directives; `'it''s'` strings, `#13` `#$0A`; `$FF` `%101` `&17` `1.5e3`; `&begin`; `asm ... end` blocks (`AsmBlocks`)
`TSynQBasicSyn`	QBasic / QB 4.5 keywords (`LEFT$`, `MID$` ...); `REM` and `'` comments; `'$INCLUDE:` / `REM $DYNAMIC`; `"strings"`; `&HFF` `&O17` `1.5E+3#` `70000&`; `name$` `count%`; line numbers and `Label:`
`TSynAnySyn`	what you give it: `Keywords`, `LineComments`, `BlockCommentStart` / `End`, `Quotes`, `DirectiveChar`, `CaseSensitive`

Each kind of token has its colours: `KeywordAttri`, `IdentifierAttri`, `NumberAttri`, `StringAttri`, `CommentAttri`, `DirectiveAttri`, `SymbolAttri`, `AsmAttri`, `LabelAttri` - each with `Foreground` and `Background` (`clDefault` : the editor's colours):

```
PasSyn.CommentAttri.Foreground := clLightGreen;
PasSyn.KeywordAttri.Foreground := clWhite;
```

`HighlighterClassForFile('MAIN.BAS')` gives the highlighter class for a file's extension (`nil` : plain text).

Speed. Only the lines on screen are coloured. `TSynLineStates` keeps the state every line starts in (inside a `{ }` comment, in an asm block), so typing re-scans the line and the next one; opening a `{` re-scans only to where the colouring settles. Paging through a 4,000 line file takes about 3 ms a page in DOSBox.

`CODEFRM.PAS` is a code editor window (the demo's Window|Code editor): File, Edit (undo, redo, clipboard, block indent), Language and Options (WordStar keys, auto indent) menus, and a status bar with the line, the column, Ins/Ovr (or `^K` / `^Q`), Modified and the language. `SYNTEST.PAS` and `EDTEST.PAS` test it all.

Timers

```
Clock := TTimer.Create(Self);           { TUIEXT }
Clock.Interval := 1000;                 { ms, in 55 ms steps (the BIOS clock) }
Clock.OnTimer := @ClockTimer;

procedure TMyForm.ClockTimer(Sender: TObject);
begin
  TimeLabel.Caption := TimeToStr(Now);
end;
```

`Interval`, `Enabled`, `OnTimer`. Timers run from the message loop - between events, also while a modal dialog is open - never in the middle of one.

Actions

An action holds what a command is: its caption, shortcut, hint, whether it can run now, and the code. Menu items, buttons and tool buttons show it.

```
Actions := TActionList.Create(Self);      { TUICTRL }
actCopy := TAction.Create(Self);
actCopy.ActionList := Actions;
actCopy.Caption := '&Copy';
actCopy.Hint := 'Copy|Copy the selection';
actCopy.ShortCut := ShortCut(Ord('C'), [ssCtrl]);
actCopy.OnExecute := @CopyExecute;
actCopy.OnUpdate := @CopyUpdate;

CopyItem := TMenuItem.Create(Self);
CopyItem.Action := actCopy;               { caption, shortcut and the click }
CopyBtn := TToolButton.Create(Self);
CopyBtn.Parent := Tools;
CopyBtn.Action := actCopy;
CopyBtn.Caption := 'Copy';                { a short caption of its own: kept }

procedure TMyForm.CopyUpdate(Sender: TObject);
begin
  actCopy.Enabled := Memo1.HasSelection;  { when idle: menu and button follow }
end;
```

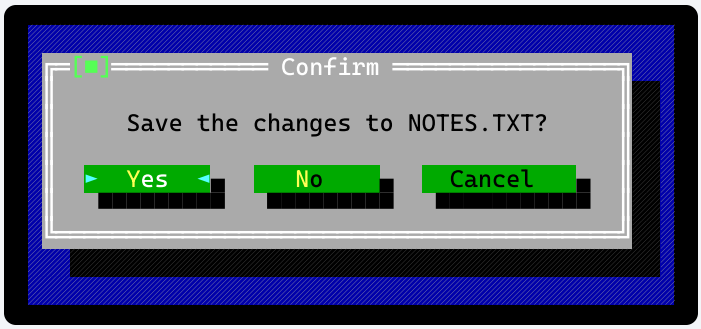
`TAction`: `Caption`, `Hint`, `ShortCut`, `Enabled`, `Checked`, `Visible`, `AutoCheck`, `OnExecute`, `OnUpdate`, `Execute`. `TActionList`: `Actions[]`, `ActionCount`, `OnUpdate`, `UpdateActions`. The shortcuts of the action lists owned by the active form work (then the main form's).

Dialogs

Message boxes (`TUISTD`):

```
ShowMessage('Saved.');

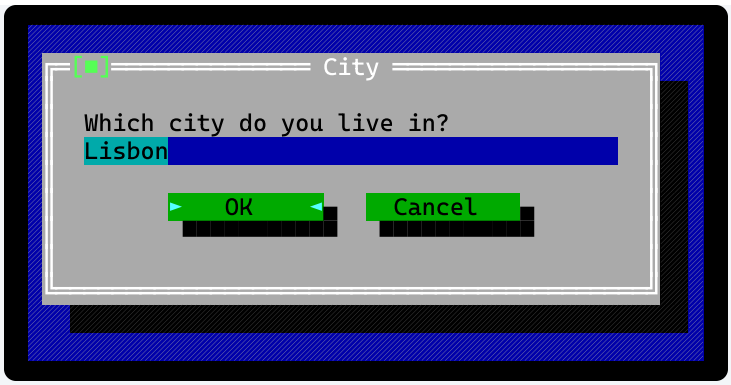
case MessageDlg('Save the changes to NOTES.TXT?', mtConfirmation,
                [mbYes, mbNo, mbCancel], 0) of
  mrYes: Save;
  mrCancel: exit;
end;
```



MessageDlg with Yes, No and Cancel.

`MessageDlg(Caption, Msg, Type, Buttons, 0)` sets the title too. Types: `mtWarning`, `mtError`, `mtInformation`, `mtConfirmation`, `mtCustom`. Buttons: `mbYes`, `mbNo`, `mbOK`, `mbCancel`, `mbAbort`, `mbRetry`, `mbIgnore`, `mbAll`, `mbNoToAll`, `mbYesToAll`, `mbClose` (and the sets `mbYesNo`, `mbYesNoCancel`, `mbOKCancel`, `mbAbortRetryIgnore`).

```
S := 'Lisbon';
if InputQuery('City', 'Which city do you live in?', S) then CityEdit.Text := S;
S := InputBox('City', 'Which city do you live in?', 'Lisbon');
```

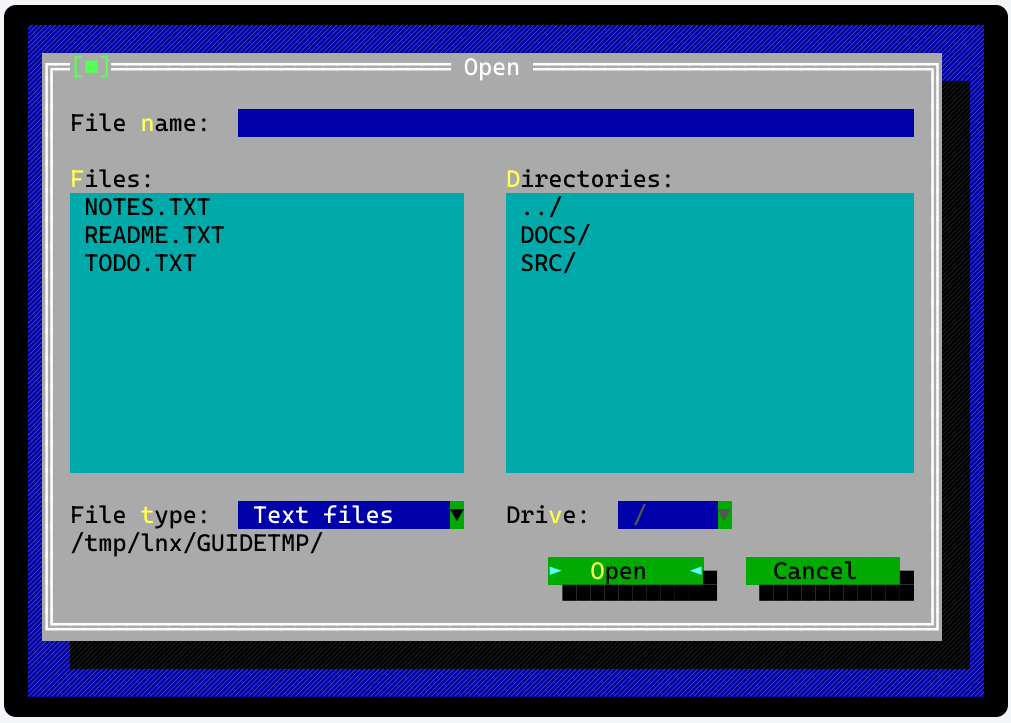


InputQuery.

File dialogs (`TUIDLGS`):

```
OpenDialog1 := TOpenDialog.Create(Self);
OpenDialog1.Filter := 'Text files|*.txt|Pascal|*.pas|All files|*.*';
OpenDialog1.Options := [ofFileMustExist];
if OpenDialog1.Execute then Memo1.Lines.LoadFromFile(OpenDialog1.FileName);

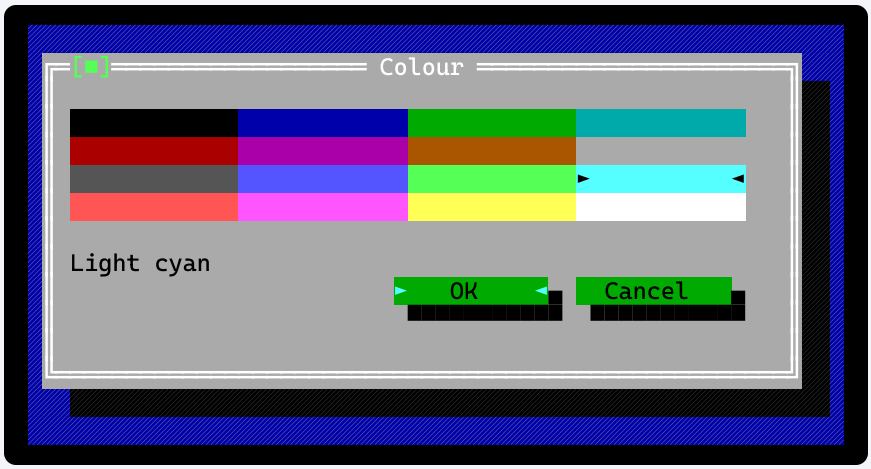
SaveDialog1 := TSaveDialog.Create(Self);
SaveDialog1.Filter := OpenDialog1.Filter;
SaveDialog1.DefaultExt := 'txt';
SaveDialog1.Options := [ofOverwritePrompt];
if SaveDialog1.Execute then Memo1.Lines.SaveToFile(SaveDialog1.FileName);
```



TOpenDialog with the Text files filter.

`FileName`, `InitialDir`, `Filter`, `FilterIndex`, `DefaultExt`, `Options` (`ofOverwritePrompt`, `ofFileMustExist`, `ofPathMustExist`), `Title`. In the dialog Enter or a double click on a folder opens it, on a file takes it; a typed folder goes there, a typed mask (`*.bak`) becomes the filter. On DOS the drive list has C: and up. `TSelectDirectoryDialog` returns a folder in `FileName`.

```
ColorDialog1 := TColorDialog.Create(Self);
ColorDialog1.Color := Memo1.Color;
if ColorDialog1.Execute then Memo1.Color := ColorDialog1.Color;
```

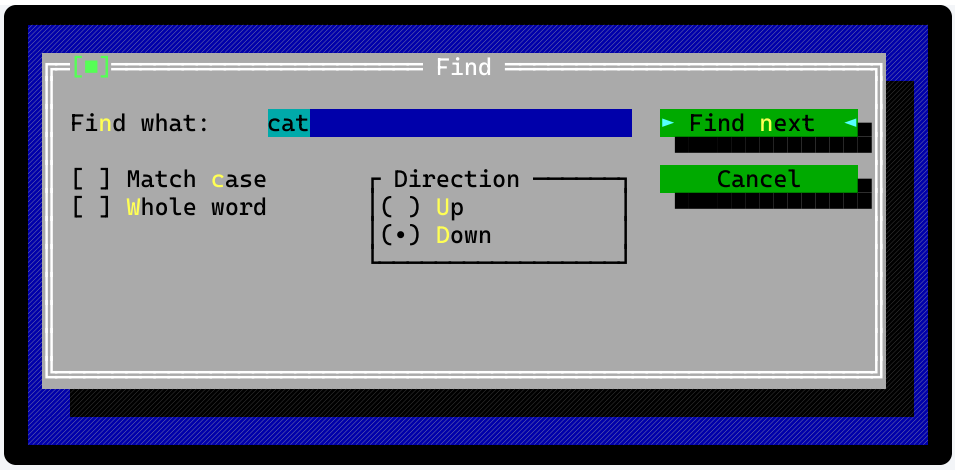


TColorDialog: the 16 colours, light cyan picked.

Find and replace stay open while you work, as in the LCL; `OnFind` and `OnReplace` do the work:

```
FindDialog1 := TFindDialog.Create(Self);
FindDialog1.Options := [frDown];
FindDialog1.OnFind := @FindDialog1Find;
FindDialog1.Execute;                    { not modal }

procedure TMyForm.FindDialog1Find(Sender: TObject);
begin
  if not MemoFindNext(Memo1, FindDialog1.FindText, FindDialog1.Options) then
    ShowMessage('Cannot find "' + FindDialog1.FindText + '"');
end;
```



TFindDialog (it stays open while you work).

`TReplaceDialog` adds `ReplaceText` and `OnReplace`; the options say which button was pressed (`frReplace`, `frReplaceAll`), and `MemoReplace(Memo1, FindText, ReplaceText, Options)` does it.

Hints

```
ShowHint := true;                       { the form: every control with a Hint }
OpenBtn.Hint := 'Open|Open a file';     { short | long }
```



A hint window under a tool button.

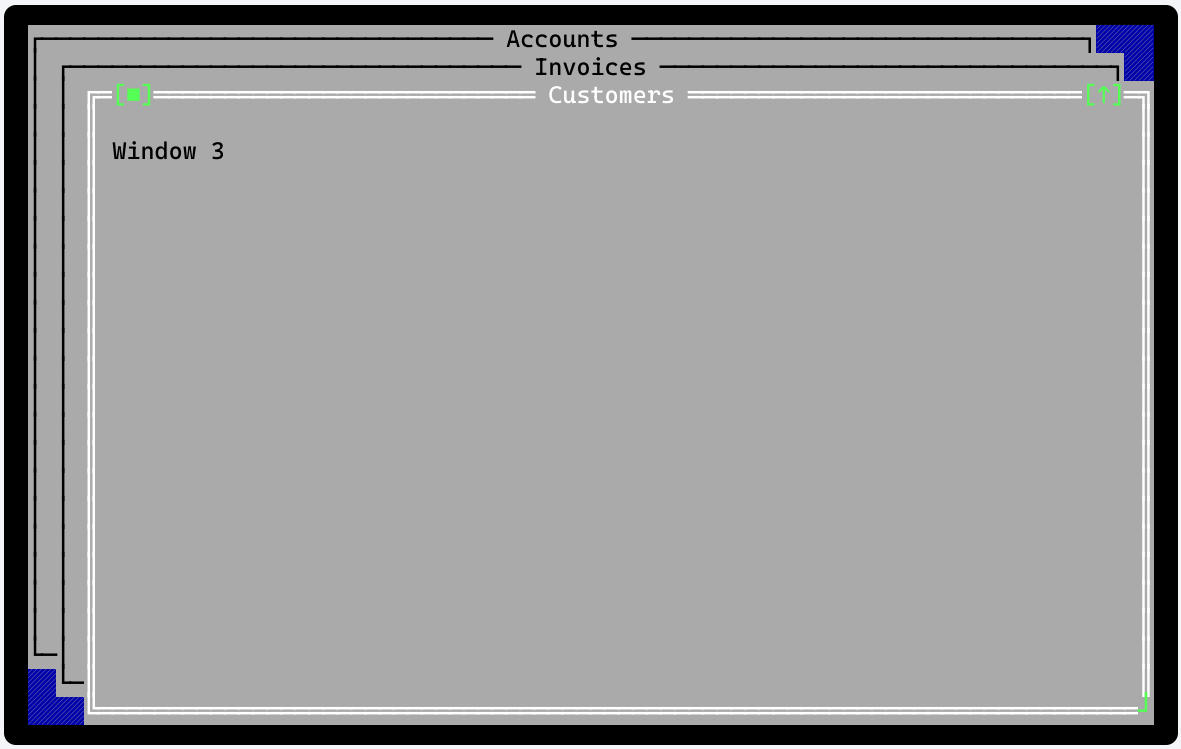
When the mouse rests on a control for `Application.HintPause` (800 ms) the short part shows under it for `HintHidePause` (5 s); a key or a move away puts it away. Status bars with `AutoHint` show the long part: the focused control's, the

tool button's under the mouse, or the menu item's. `Application.ShowHint := false` switches hint windows off.

The application

Windows. F6 and Shift+F6 go to the next and previous window. `Application.Tile` and `Application.Cascade` arrange the visible windows in the desktop; `Application.Forms[]`, `Screen.Forms[]`, `Screen.FormCount`, `Screen.ActiveForm` list them.

```
Application.Cascade;
```



Application.Cascade (the whole screen).

```
Application.Tile;
```



Application.Tile (the whole screen).

Text modes. Set `TUITextCols` / `TUITextRows` before `Application.Initialize`, or switch while running:

```
if not Application.SetTextMode(132, 50) then
  Application.SetTextMode(80, 50);
```

80 x 25/28/43/50 work on VGA; 80 x 60 and 132 x 25/43/50/60 need a VESA text mode (or the S3/Cirrus 132 x 25 and 132 x 43 modes, which DOSBox has). The screen bars move, forms stay on the screen, maximized forms fill the new desktop, then `Application.OnScreenResize` runs.

The clipboard. `Clipboard.AsText` (inside the program); edits and memos use it for Ctrl+C/X/V.

Exceptions. An exception in an event shows an error box and the program goes on; `Application.OnException` replaces the box.

The mouse wheel (CuteMouse 2 and other drivers with the wheel API): lists, memos, grids, views and scroll boxes scroll; scroll bars, track bars and spin edits move. `OnMouseWheel` on any control can take it first.

Idle, loops, tests. `Application.OnIdle` runs when there is nothing to do. `Application.ProcessMessages` handles what is waiting. `TUIPostKey`, `TUIPostChar`, `TUIPostString`, `TUIPostMouse`, `TUIPostClick` and `TUIPostWheel` feed input as if typed or clicked; `TUITEST.PAS` uses them.

Speed. Everything is drawn into a buffer of the screen; once per loop only the cells that changed go to video memory. Repainting a whole form that did not change writes nothing.

Keys

Key	Does
Tab, Shift+Tab	next / previous control
arrows	in a control; else the next / previous control
Alt+letter	the control (or menu, or tab) with that `&` letter
letter	the same, when the focus is not in a text field
Enter	the focused button, else the `Default` button
Esc	the `Cancel` button, else closes a modal form
F10	the menu bar (the form's, else the screen's)
Shift+F10	the focused control's pop-up menu
F6, Shift+F6	next / previous window
Ctrl+F5	move / size the window: arrows, Shift+arrows, Z, Enter
Alt+F4	close the window
Ctrl+C, Ctrl+X, Ctrl+V	copy, cut, paste (also Ctrl+Ins, Shift+Del, Shift+Ins)
Ctrl+A	select all
Ins	insert / overwrite in text fields
Alt+Down, F4	open a combo box's list
Ctrl+Tab, Ctrl+PgDn/PgUp	next / previous page of a page control

LCL unit names

The `LCL\` folder has units with the LCL's names - `Forms`, `Controls`, `Graphics`, `StdCtrls`, `ExtCtrls`, `ComCtrls`, `Menus`, `Dialogs`, `Grids`, `Spin`, `MaskEdit`, `CheckLst`, `ActnList`, `Buttons`, `Clipbrd`, `LCLType` - so a form unit can start with Lazarus's own uses clause:

```
uses
  Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls, ExtCtrls;
```

Build with `-FuLCL`, and keep that folder apart from FPTEGL's units of the same names. `LCLFORM.PAS` and `LCLTEST.PAS` show it; `MKLCL.PY` remakes the folder after a TUI unit's interface changes.

Which unit has what

Unit	Has
`TUICTRL`	`TControl`, `TWinControl`, `TForm`, `TApplication` (`Application`), `TScreen` (`Screen`), `TCanvas`, `TAction`, `TActionList`, `Clipboard`, `TColor` and `cl...`, `VK_...`, `mr...`, `ShortCut`, the `.lfm` reader (`RegisterFormText`, `RegisterFormFile`, `ReadFormText`, `ReadFormFile`, `LfmWarnings`, `LfmStrict`)
`TUISTD`	`TLabel`, `TButton`, `TEdit`, `TCheckBox`, `TRadioButton`, `TGroupBox`, `TPanel`, `ShowMessage`, `MessageDlg`, `InputQuery`, `InputBox`
`TUILIST`	`TListBox`, `TComboBox`
`TUIMEMO`	`TMemo`
`TUIMENU`	`TMainMenu`, `TMenuItem`, `TPopupMenu`, `NewItem`, `NewSubMenu`, `NewLine`, `TStatusBar`
`TUIEXT`	`TTimer`, `TScrollBar`, `TCheckListBox`, `TRadioGroup`, `TSpinEdit`, `TMaskEdit`, `TProgressBar`, `TStaticText`, `TBevel`
`TUIPAGES`	`TPageControl`, `TTabSheet`, `TScrollBox`, `TSplitter`, `TSpeedButton`, `TToolBar`, `TToolButton`
`TUIGRID`	`TStringGrid`, `TDrawGrid`
`TUIVIEWS`	`TListView`, `TTreeView`, `TTrackBar`

Unit	Has
`TUISYN`	`TSynPasSyn`, `TSynQBasicSyn`, `TSynAnySyn`, `TSynLineStates`, `HighlighterClassForFile`
`TUISYNED`	`TSynEdit`
`TUIDLGS`	`TOpenDialog`, `TSaveDialog`, `TSelectDirectoryDialog`, `TColorDialog`, `TFindDialog`, `TReplaceDialog`, `MemoFindNext`, `MemoReplace`
`TUISCRN`	the screen buffer and drawing (`ScrPutStr`, `ScrBox` ...), used by `TCanvas`
`TUIHW`	the hardware: video, keyboard, mouse, text modes

Tool	Does
`LFM2INC.PAS`	an `.lfm` into an include file for `RegisterFormText`, see LFM2INC
`MKLCL.PY`	remakes the `LCL\` units after a TUI unit's interface changes
`GUIDESHOT.PAS`, `MKGUIDE.PY`	remake this guide's screens and the guide itself